PRODUCT REQUIREMENT DOCUMENT

NeoBank - Digital Banking Platform

Version 2.0 | Hackathon Edition | March 2026

🎯 Full Stack Hackathon Project

📋 Table of Contents

1. Executive Summary
2. Problem Statement
3. Proposed Solution
4. Tech Stack
5. Hackathon Requirements
6. Product Features
7. Implementation Roadmap
8. Conclusion

► 1. EXECUTIVE SUMMARY

NeoBank is a comprehensive digital banking platform designed to revolutionize how individuals manage their finances. In an era where traditional banking often falls short in delivering seamless, user-centric experiences, NeoBank emerges as a modern solution that combines cutting-edge technology with intuitive design.

This project is developed as a **Full Stack Hackathon submission**, implementing all mandatory hackathon requirements while solving real-world banking challenges. The application demonstrates proficiency in React, Tailwind CSS, Node.js, Express.js, and MongoDB.

► 2. PROBLEM STATEMENT

Challenges in Traditional Banking

- ✗ **Complex User Interfaces** - Legacy banking apps are cluttered and unintuitive
- ✗ **Limited Accessibility** - Traditional banks operate within restricted hours
- ✗ **Slow Transactions** - Fund transfers often take days to complete
- ✗ **Poor Financial Visibility** - Customers struggle to see complete financial health
- ✗ **Security Concerns** - Increasing cyber threats demand better protection
- ✗ **Hidden Fees** - Complicated fee structures confuse customers

► 3. PROPOSED SOLUTION

How NeoBank Addresses These Challenges

- ✓ Modern, intuitive interface built with **React & Tailwind CSS**
- ✓ 24/7 cloud-native accessibility
- ✓ Real-time transactions with instant balance updates
- ✓ Unified financial dashboard
- ✓ Bank-grade security with JWT authentication
- ✓ Transparent operations with clear fee structure

► 4. TECHNOLOGY STACK

✅ **Required Hackathon Tech Stack - Fully Implemented**

Category	Technology	Implementation Status
Frontend	ReactJS	✓ **Implemented**

Category	Technology	Implementation Status
Styling	Tailwind CSS	✓ **Implemented**
Backend	Node.js	✓ **Implemented**
Framework	Express.js	✓ **Implemented**
Database	MongoDB	✓ **Implemented**

◆ Frontend Architecture

React 18+ with Hooks, React Router, Context API, Axios

◆ Backend Architecture

Node.js + Express REST API, Mongoose ODM, JWT Auth

► 5. HACKATHON REQUIREMENTS CHECKLIST

All 13 mandatory requirements from the hackathon event:

#	Requirement	Implementation Details	Status
1	**Routing & Navigation**	React Router with pages: Home, Login, Signup, Dashboard, Profile	✓
2	**React Hooks**	useState, useEffect, useRef, useContext used throughout	✓
3	**State Management**	Context API for auth state, global settings, theme	✓
4	**Authentication**	Signup, Login, JWT tokens, LocalStorage, Protected routes	✓
5	**Theme Support**	Dark/Light mode toggle with persistence	✓
6	**Search, Filter, Sort**	Transaction search, category filter, date sorting	✓
7	**Debouncing**	Search bar with debounced API calls	✓
8	**Pagination**	Backend pagination (limit/skip) with UI controls	✓

#	Requirement	Implementation Details	Status
9	**CRUD Operations**	Create, Read, Update, Delete for accounts & transactions	✓
10	**API Integration**	REST APIs with Express, error handling, loading states	✓
11	**Form Validation**	Input validation, error messages, controlled components	✓
12	**Responsive UI**	Tailwind CSS for mobile, tablet, desktop	✓
13	**Error Handling**	Backend error responses, frontend error display, try-catch	✓

► 6. PRODUCT FEATURES

6.1 Authentication & Security CORE

◆ User Registration useState

Secure signup with email validation and password strength requirements.

◆ User Login useEffect

Email/password authentication with JWT token generation.

◆ JWT Authentication Context API

Stateless token-based auth with HTTP-only cookies & LocalStorage.

◆ Protected Routes React Router

Private routes for Dashboard, Profile requiring authentication.

6.2 Account Management CORE

◆ Dashboard useContext

Real-time account balance, recent transactions overview.

Profile Management

Update personal info, contact details, preferences.

Account Statements

View and filter transaction history.

Settings

Theme toggle, notification preferences.

6.3 Transactions & Operations CORE

Fund Transfer CRUD

Transfer funds between accounts (Create operation).

Transaction History Read

Paginated transaction list with filters (Read operation).

Edit Transaction Update

Modify transaction details (Update operation).

Delete Transaction Delete

Remove unwanted transactions (Delete operation).

6.4 Search, Filter & Pagination ENHANCED

Search Debounce

Search transactions by description with 300ms debounce.

Filter

Filter by category, date range, transaction type.

Sort

Sort by amount, date (ascending/descending).

Pagination

Backend pagination with 10 items per page, page controls.

6.5 UI/UX Features ENHANCED

Dark/Light Mode useState

Theme toggle with localStorage persistence.

Loading States API

Skeleton loaders and spinners during API calls.

Error Handling

Toast notifications for success/error feedback.

Responsive Design

Mobile-first Tailwind CSS implementation.

7. IMPLEMENTATION ROADMAP

1 Phase 1: Foundation (Week 1)

- Project setup: React + Tailwind + Express

- Backend: MongoDB connection, user model
- Auth: Registration, Login APIs with JWT
- Frontend: React Router setup, Auth pages
- **Delivers:** Requirements 1, 2, 3, 4, 11

2 Phase 2: Core Features (Week 2)

- Dashboard with account overview
- Transaction CRUD operations
- Protected routes implementation
- Context API for global state
- **Delivers:** Requirements 9, 10, 13

3 Phase 3: Search & Pagination (Week 3)

- Search functionality with debouncing
- Filter and sort options
- Backend pagination (limit/skip)
- Pagination UI controls
- **Delivers:** Requirements 6, 7, 8

4 Phase 4: UI Polish & Theme (Week 4)

- Dark/Light mode toggle
- Theme persistence in localStorage
- Responsive design optimization

- Loading states & error handling
- **Delivers:** Requirements 5, 12

► 8. SUCCESS METRICS

13/13	100%	100%	4
Requirements Met	Tech Stack Coverage	Responsive	Week Timeline

► 9. CONCLUSION

NeoBank is a complete full-stack banking application built for the Full Stack Hackathon. It demonstrates mastery of React, Tailwind CSS, Node.js, Express.js, and MongoDB while implementing all 13 mandatory hackathon requirements.

🚀 **Hackathon Readiness**

- ✓ All required tech stack implemented
- ✓ All 13 mandatory features completed
- ✓ Real-world problem solved
- ✓ Production-ready code structure